

STRING_REFERENCE

Core Definition

<string> provides `std::basic_string` and its aliases (`string`, `wstring`, `u8string`, `u16string`, `u32string`) — owning, dynamically-sized, null-terminated character sequences with value semantics. It also supplies the numeric conversion functions (`stoi`, `to_string`) and the `"s"` literal.

#CPP #STRING #STD-STRING #TEXT #STRING-VIEW #SUBSTR #FIND #CONVERSION

TYPE ALIASES

```
std::string      = std::basic_string<char>           // The everyday one
std::wstring     = std::basic_string<wchar_t>
std::u8string    = std::basic_string<char8_t>        // C++20
std::u16string   = std::basic_string<char16_t>       // C++11
std::u32string   = std::basic_string<char32_t>       // C++11
std::pmr::string =                                // C++17: polymorphic allocator variant
```

COMPLETE STRING QUICK REFERENCE

MEMBER FUNCTIONS — Target | Operation | Output

```
// =====
// CONSTRUCTION
// =====
std::string s;                // none | Empty string | size() == 0
std::string s("hello");       // const char* | From C-string | Stops at '\0'
std::string s("hello", 3);    // ptr + count | First n chars | "hel"
std::string s(5, 'x');        // count + char | Repeated char | "xxxxx"
std::string s(other);         // string | Copy | Deep copy
std::string s(std::move(other)); // rvalue | Move | other left valid but
    unspecified
std::string s(other, pos);    // string + pos | Substring from pos | Throws out_of_range if pos >
    size
std::string s(other, pos, len); // + length | Substring | len clamped to remaining
std::string s(first, last);    // iterators | From a range | Any char iterator pair
std::string s({'a','b','c'});  // init list | From initializer | "abc"
std::string s(sv);             // string_view | From a view | C++17, explicit
std::string s = "text"s;       // literal | UDL | using namespace
    std::string_literals

// =====
// ELEMENT ACCESS
// =====
s[i]                // index | Char at i | NO bounds check; s[size()] is
    '\0'
s.at(i)             // index | Char at i, checked | Throws std::out_of_range
s.front()           // none | First char | UB if empty
s.back()            // none | Last char | UB if empty
s.data()            // none | Pointer to buffer | C++11+: null-terminated;
    C++17+: non-const overload
s.c_str()           // none | Null-terminated ptr | const char*, for C APIs

// =====
// CAPACITY
```

s.size()	// none	Number of chars	Returns size_t
s.length()	// none	Same as size()	Returns size_t
s.empty()	// none	size() == 0	Returns bool
s.capacity()	// none	Allocated chars	Returns size_t (>= size)
s.reserve(n)	// count	Ensure capacity	Returns void; avoids
reallocation			
s.shrink_to_fit()	// none	Release excess	Non-binding request
s.max_size()	// none	Theoretical limit	Returns size_t
s.resize(n)	// count	Change size	Pads with '\0'
s.resize(n, ch)	// count + char	Change size, pad ch	Returns void
s.resize_and_overwrite(n, op)	// C++23	Resize + fill via op	Avoids zero-init cost
// MODIFIERS – APPEND & INSERT			
s += other	// string/char/ptr	Append	Returns string&
s.append(str)	// string	Append	Returns string&
s.append(str, pos, len)	// + range	Append substring	Returns string&
s.append(cstr)	// const char*	Append C-string	Returns string&
s.append(cstr, n)	// ptr + count	Append n chars	Returns string&
s.append(n, ch)	// count + char	Append n copies	Returns string&
s.append(first, last)	// iterators	Append range	Returns string&
s.push_back(ch)	// char	Append one char	Returns void
s.insert(pos, str)	// index+string	Insert at index	Returns string&
s.insert(pos, n, ch)	// + count/char	Insert n copies	Returns string&
s.insert(it, ch)	// iterator	Insert before it	Returns iterator
s.insert(it, first, last)	// iter + range	Insert range	Returns iterator
// MODIFIERS – ERASE, REPLACE, CLEAR			
s.clear()	// none	Remove all chars	Capacity unchanged
s.erase()	// none	Erase all	Returns string&
s.erase(pos)	// index	Erase from pos	Returns string&
s.erase(pos, len)	// index + len	Erase len chars	Returns string&
s.erase(it)	// iterator	Erase one char	Returns iterator
s.erase(first, last)	// iterators	Erase range	Returns iterator
s.pop_back()	// none	Remove last char	UB if empty
s.replace(pos, len, str)	// range+string	Replace substring	Returns string&
s.replace(pos, len, str, pos2, len2)	// + sub-range	Replace with part	Returns string&
s.replace(pos, len, n, ch)	// + count/char	Replace with chars	Returns string&
s.replace(first, last, str)	// iterators	Replace range	Returns string&
std::erase(s, ch)	// C++20 free fn	Erase all ch	Returns count removed
std::erase_if(s, pred)	// C++20 free fn	Erase matching	Returns count removed
// SUBSTRING & COPY			
s.substr(pos)	// index	From pos to end	Returns NEW string; throws if
pos > size			
s.substr(pos, len)	// index + len	len chars from pos	Returns NEW string; len
clamped			
s.copy(buf, len, pos)	// ptr+len+pos	Copy into buffer	Returns chars copied; NO '\0'
added			
s.swap(other)	// string&	Exchange contents	O(1), no allocation
// SEARCHING – returns index or std::string::npos			
s.find(str[, pos])	// needle[+start]	First occurrence	Returns size_t or npos
s.rfind(str[, pos])	// needle[+start]	LAST occurrence	Searches backward from pos
s.find_first_of(chars[, pos])	// char set	First char in set	Returns index or npos
s.find_last_of(chars[, pos])	// char set	Last char in set	Returns index or npos
s.find_first_not_of(chars[, pos])	// char set	First char NOT in	Returns index or npos
s.find_last_not_of(chars[, pos])	// char set	Last char NOT in	Returns index or npos
All accept: string, string_view, const char*, char			
std::string::npos	// constant	"not found"	static const size_t = -1
// SEARCHING – C++20 PREDICATES			

```

s.starts_with(x)           // sv/char/cstr | Prefix test      | Returns bool (C++20)
s.ends_with(x)             // sv/char/cstr | Suffix test        | Returns bool (C++20)
s.contains(x)              // sv/char/cstr | Substring test     | Returns bool (C++23)

// =====
// COMPARISON
// =====
a == b, a != b, a < b, ... // operators      | Lexicographic      | Returns bool; works with const
    char* too
a <=> b                    // C++20          | Three-way compare  | Returns strong_ordering
s.compare(str)             // string         | Full compare       | Returns <0, 0, >0
s.compare(pos, len, str)   // range          | Compare substring  | Returns int
s.compare(pos, len, str, pos2, len2) // both ranges | Compare substrings | Returns int

// =====
// ITERATORS
// =====
s.begin() / s.end()        // Forward iterators
s.cbegin() / s.cend()      // Const forward iterators
s.rbegin() / s.rend()      // Reverse iterators
s.crbegin() / s.crend()    // Const reverse iterators
for (char c : s) { }       // Range-for works directly

// =====
// NON-MEMBER FUNCTIONS
// =====
a + b                      // string+any       | Concatenate        | Returns new string
os << s                    // ostream          | Write string        | Returns ostream&
is >> s                    // istream          | Read one WORD       | Skips leading ws, stops at ws
std::getline(is, s)        // istream          | Read a LINE         | Discards '\n'
std::getline(is, s, delim) // + delimiter      | Read to delimiter   | Returns istream&
std::swap(a, b)            // two strings      | Swap                | 0(1)
std::hash<std::string>{}(s) // string           | Hash value          | For unordered containers

// =====
// NUMERIC CONVERSIONS – string → number
// =====
std::stoi(s[, &pos][, base]) // string           | → int              | Throws invalid_argument /
    out_of_range
std::stol(s[, &pos][, base]) // string           | → long             | pos receives chars consumed
std::stoll(s[, &pos][, base]) // string           | → long long        | base 0 = auto-detect 0x/0
std::stoul(s[, &pos][, base]) // string           | → unsigned long    | Note: accepts "-1" and wraps
std::stoull(s[, &pos][, base]) // string           | → unsigned long long
std::stof(s[, &pos])        // string           | → float
std::stod(s[, &pos])        // string           | → double
std::stold(s[, &pos])       // string           | → long double

// =====
// NUMERIC CONVERSIONS – number → string
// =====
std::to_string(v)           // arithmetic       | → string            | int/long/long
    long/unsigned/float/double
std::to_wstring(v)          // arithmetic       | → wstring           |
// <charconv> (C++17): locale-independent, non-allocating, fastest
std::from_chars(first, last, value[, base]) // → from_chars_result {ptr, ec}
std::to_chars(first, last, value[, fmt[, prec]]) // → to_chars_result {ptr, ec}
// <format> (C++20)
std::format("{} ", v)        // any formattable | → string            | Type-safe, extensible

// =====
// LITERALS
// =====
using namespace std::string_literals;
"text"s                     // → std::string      (C++14)
u8"text"s / u"text"s / U"text"s / L"text"s // → u8string / u16string / u32string / wstring
using namespace std::string_view_literals;
"text"sv                    // → std::string_view (C++17)

```

std::string_view — <string_view> (C++17)

```

std::string_view sv = s;           // Non-owning view; no allocation, no copy
std::string_view sv("literal");    // 0(1) construction from a literal
std::string_view sv(ptr, len);     // From pointer + length

sv.size() / sv.empty() / sv[i] / sv.at(i) / sv.front() / sv.back() / sv.data()
sv.substr(pos[, len])              // Returns ANOTHER VIEW – no allocation
sv.remove_prefix(n)                 // Shrink from the front | Returns void
sv.remove_suffix(n)                 // Shrink from the back  | Returns void
sv.find / rfind / find_first_of / ... // Same search interface as string
sv.starts_with / ends_with          // C++20
sv.contains                         // C++23
sv.compare(other)                   // Returns int
std::string(sv)                     // Explicit copy into an owning string

// CAUTION: data() is NOT guaranteed null-terminated. Never pass sv.data() to a C API.
// CAUTION: the view does not own – never outlive the underlying buffer.

```

COMMON PATTERNS & EXAMPLES

Building Strings Efficiently

```

#include <string>
#include <sstream>

// Repeated concatenation – reserve to avoid repeated reallocation
std::string build(const std::vector<std::string>& parts) {
    std::size_t total = 0;
    for (const auto& p : parts) total += p.size() + 1;

    std::string out;
    out.reserve(total);
    for (const auto& p : parts) { out += p; out += ' '; }
    return out;
}

// Mixed types – ostream or std::format
std::ostringstream ss;
ss << "id=" << 42 << " score=" << 3.14;
std::string msg = ss.str();

// C++20 – clearest of all:
// std::string msg = std::format("id={} score={:.2f}", 42, 3.14);

```

Splitting

```

#include <string>
#include <string_view>
#include <vector>
#include <sstream>

// Whitespace split – simplest
std::vector<std::string> splitWS(const std::string& s) {
    std::istringstream ss(s);
    std::vector<std::string> out;
    std::string word;
    while (ss >> word) out.push_back(word);
    return out;
}

// Delimiter split, keeping empty fields

```

```

std::vector<std::string> split(const std::string& s, char delim) {
    std::vector<std::string> out;
    std::istringstream ss(s);
    std::string item;
    while (std::getline(ss, item, delim)) out.push_back(item);
    return out;
}

// Zero-allocation split with string_view
std::vector<std::string_view> splitView(std::string_view s, char delim) {
    std::vector<std::string_view> out;
    std::size_t start = 0;
    while (true) {
        auto pos = s.find(delim, start);
        out.push_back(s.substr(start, pos - start));
        if (pos == std::string_view::npos) break;
        start = pos + 1;
    }
    return out;
}

```

Joining

```

std::string join(const std::vector<std::string>& parts, std::string_view sep) {
    std::string out;
    for (std::size_t i = 0; i < parts.size(); ++i) {
        if (i) out += sep;
        out += parts[i];
    }
    return out;
}

```

Trimming

```

#include <string>

const char* WS = " \t\n\r\f\v";

std::string ltrim(std::string s) {
    s.erase(0, s.find_first_not_of(WS));
    return s;
}

std::string rtrim(std::string s) {
    s.erase(s.find_last_not_of(WS) + 1); // npos + 1 == 0 → erases everything
    return s;
}

std::string trim(std::string s) { return ltrim(rtrim(std::move(s))); }

```

Case Conversion

```

#include <algorithm>
#include <cctype>

std::string toLower(std::string s) {
    std::transform(s.begin(), s.end(), s.begin(),
        [](unsigned char c) { return std::tolower(c); });
    return s;
}

// The unsigned char cast is REQUIRED – passing a negative char to tolower is UB.
// Note: this is ASCII-only. Unicode case folding needs ICU or similar.

```

Replace All Occurrences

```
std::string replaceAll(std::string s, std::string_view from, std::string_view to) {
    if (from.empty()) return s;
    std::size_t pos = 0;
    while ((pos = s.find(from, pos)) != std::string::npos) {
        s.replace(pos, from.size(), to);
        pos += to.size();           // Skip the replacement, avoids infinite loop
    }
    return s;
}
```

Searching Idioms

```
std::string s = "hello world";

if (s.find("world") != std::string::npos) { /* found */ }

// C++20 / C++23 - clearer:
if (s.starts_with("hello")) { }
if (s.ends_with("world")) { }
if (s.contains("lo w")) { } // C++23

// Iterate every occurrence
for (std::size_t p = s.find('o'); p != std::string::npos; p = s.find('o', p + 1)) {
    // p is each index of 'o'
}
```

Safe Numeric Conversion

```
#include <string>
#include <charconv>
#include <optional>

// Exception-based (std::stoi)
std::optional<int> parseInt(const std::string& s) {
    try {
        std::size_t pos;
        int v = std::stoi(s, &pos);
        if (pos != s.size()) return std::nullopt; // Trailing junk
        return v;
    } catch (const std::exception&) {
        return std::nullopt; // invalid_argument or out_of_range
    }
}

// Non-throwing, locale-independent, fastest (C++17)
std::optional<int> parseIntFast(std::string_view s) {
    int v{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{} || ptr != s.data() + s.size()) return std::nullopt;
    return v;
}
```

`std::stoi("12abc")` returns 12 without complaint. Check `pos` if trailing characters matter.

Reading Input

```
std::string word, line;

std::cin >> word;           // One whitespace-delimited token
std::getline(std::cin, line); // Whole line

// The classic mix-up:
int n;
std::cin >> n;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // Required
std::getline(std::cin, line);
```

Passing Strings to Functions

```
// Read-only, may receive literals, string, or string_view – no allocation
void log(std::string_view msg);

// Needs an owning copy (stores it) – take by value and move
void store(std::string s) { data_ = std::move(s); }

// Needs a null-terminated C string for a C API
void callC(const std::string& s) { ::c_api(s.c_str()); }

// Modifies in place
void normalize(std::string& s);
```

Small String Optimization (SSO)

```
std::string tiny = "short"; // Typically stored inline – NO heap allocation
std::string big(100, 'x');  // Heap allocated
// libstdc++/libc++ inline capacity is ~15 chars for std::string on 64-bit.
// This is why passing short strings by value is cheaper than intuition suggests.
```

Interop with C

```
std::string s = "path/to/file";

std::FILE* f = std::fopen(s.c_str(), "r"); // c_str(): guaranteed '\0'
// NOT sv.data() – string_view has no terminator guarantee

// C buffer → string
char buf[256];
std::size_t n = ::read_into(buf, sizeof(buf));
std::string result(buf, n); // Explicit length; handles embedded '\0'
```

Embedded Null Characters

```
std::string s("a\0b", 3); // size() == 3 – the '\0' is real data
std::string t("a\0b");    // size() == 1 – stops at the first '\0'

// std::string handles embedded nulls; C-string functions do not.
```

IMPORTANT CONCEPTS

`npos` Is the Sentinel

`std::string::npos` is `static_cast<size_t>(-1)` — the largest `size_t`. Always compare `find()` results against it explicitly; never test for `> 0` or `!= 0`, since index 0 is a valid match.

`substr` Copies; `string_view::substr` Doesn't

`s.substr(a, b)` allocates a new string. In hot loops, view-based slicing avoids that entirely — at the cost of lifetime responsibility.

`string_view` Lifetime Is Your Job

```
std::string_view bad() {
    std::string tmp = "danger";
    return tmp;           // DANGLING — tmp dies at return
}
std::string_view alsoBad = std::string("x") + "y"; // Temporary destroyed immediately
```

A view is a pointer and a length. Never store one longer than the buffer it points into.

`operator[]` VS `at()`

`[]` is unchecked (fast); `at()` throws `std::out_of_range`. `s[s.size()]` is defined and returns `'\0'`; any further index is UB.

Iterator and Reference Invalidation

Any operation that may reallocate — `push_back`, `append`, `+=`, `insert`, `resize`, `reserve` — invalidates all iterators, pointers and references into the string. `c_str()` / `data()` pointers become stale the same way.

Prefer `reserve` in Build Loops

Without it, appending in a loop reallocates logarithmically often, copying every time. One `reserve` with a good estimate removes all of it.

`stoul` Accepts Negative Input

`std::stoul("-1")` returns `ULONG_MAX` rather than throwing — the underlying `strtoul` wraps. Validate signs yourself when parsing unsigned values.

Conversions Are Locale-Sensitive

`std` and `to_string` honor the global C locale, which means a comma decimal separator in some locales. `<charconv>`'s `from_chars` / `to_chars` are locale-independent and belong in serialization code.

`to_string(double)` Is Fixed at 6 Decimals

It behaves like `printf("%f")`, so `to_string(0.1)` gives `"0.100000"` and large values lose precision. Use `std::format("{}, v)", v)` or `to_chars` for round-trippable output.

`std::string` Is Not Unicode-Aware

`size()` counts `char` units (bytes), not code points or grapheme clusters. Indexing into UTF-8 mid-sequence splits a character. For real text processing use ICU, or at minimum keep operations at sequence boundaries.

C++23 `resize_and_overwrite`

Fills a buffer without paying for the zero-initialization `resize` normally performs — useful when handing the buffer to a C API that writes into it.

PERFORMANCE NOTES

Operation	Complexity	Note
<code>size()</code> , <code>empty()</code> , <code>operator[]</code>	$O(1)$	
<code>push_back</code> / <code>+=</code> (amortized)	$O(1)$	Reallocates geometrically
<code>insert</code> / <code>erase</code> in the middle	$O(n)$	Shifts the tail
<code>find</code> (naive)	$O(n \cdot m)$	No preprocessing
<code>substr</code>	$O(\text{len})$	Allocates
<code>string_view::substr</code>	$O(1)$	No allocation
<code>swap</code>	$O(1)$	Pointer exchange
<code>comparison</code>	$O(n)$	
SSO short strings	no allocation	~15 chars typical on 64-bit

BEST PRACTICES

1. Take read-only string parameters as `std::string_view` (C++17) — accepts everything, allocates nothing
2. Take by value and `std::move` when the function stores the string
3. `reserve()` before building in a loop
4. Always compare `find()` against `npos`
5. Use `c_str()` for C APIs, never `string_view::data()`
6. Cast to `unsigned char` before calling any `<cctype>` function
7. Prefer `starts_with` / `ends_with` / `contains` (C++20/23) over `find`-based idioms
8. Use `<charconv>` for serialization — locale-independent and fast
9. Prefer `std::format` over `ostringstream` chains in new code
10. Never let a `string_view` outlive its buffer
11. Don't index UTF-8 by byte and expect characters
12. Use `std::move` on strings you are done with — moves are $O(1)$ for heap-allocated ones
13. `empty()`, not `size() == 0` — clearer, and guaranteed $O(1)$
14. Watch for iterator invalidation after any growth operation

RELATED HEADERS

```
#include <string>           // std::string, stoi, to_string, ""s
#include <string_view>       // C++17 std::string_view, ""sv
#include <charconv>          // C++17 from_chars / to_chars – fast, locale-free
#include <format>            // C++20 std::format
#include <print>             // C++23 std::print / println
#include <sstream>           // istream / ostream
#include <cstring>           // C-string functions (interop)
#include <cctype>            // isalpha, tolower (pass unsigned char)
#include <algorithm>         // transform, find, sort, reverse
#include <regex>             // pattern matching
#include <locale>            // locale-aware conversion facets
```

EXTERNAL RESOURCES

- `std::basic_string`: https://en.cppreference.com/w/cpp/string/basic_string
 - `std::string_view`: https://en.cppreference.com/w/cpp/string/basic_string_view
 - `<charconv>`: <https://en.cppreference.com/w/cpp/header/charconv>
 - **Core Guidelines, strings**: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#SS-string>
-

Standard: C++11 (stoi, move) / C++14 ("s) / C++17 (string_view, charconv) / C++20 (starts_with, format) / C++23 (contains, resize_and_overwrite) **Last Updated:** September 2026